

Doc. no.:	P1652R0
Date:	2019-05-27
Audience:	LEWG
Reply-to:	Zhihao Yuan <zy@miator.net> Victor Zverovich <victor.zverovich@gmail.com>

Printf corner cases in `std::format`

Introduction

Printf heavily influences the formatting behavior design of `std::format` and Python `str.format`. However, in the process of development, the current specification of `std::format` ^[1] misses a few beneficial outcomes comparing to `printf` and Python but inherits some unnecessary compromise from `iostreams`. This document is to show these corner cases and propose solutions in C++20.

Problem 1: ‘#o’ specification should not print 0 as “00”

variant	behavior
printf	#o and #x print “0” for 0
Python	#o, #x, and #b print “0o0”, “0x0”, “0b0”, respectively, for 0
format	#o, #x, and #b print “00”, “0x0”, “0b0”, respectively, for 0

`0odddd` is not a pattern for octal literals in C++, so `std::format` replaces it with `printf`’s pattern `dddd` for `#o`. However, the `#` flag in `printf` is specified as follows^[2]:

For `o` conversion, it increases the precision, if and only if necessary, to force the first digit of the result to be a zero (if the value and precision are both 0, a single 0 is printed).

The output here matches C++ syntax where 0 is an octal literal. We propose to respecify `std::format` ‘#o’ to match `printf` output.

Before:

```
std::string s = std::format("{:#o}", 0); // s == "00"
```

After:

```
std::string s = std::format("{:#o}", 0); // s == "0"
```

Problem 2: ‘c’ should be able to print 65 as “A” (ASCII)

variant	behavior
printf	‘c’ prints “A” for 65, ‘lc’ prints “A” for (wint_t) 65
Python	‘c’ prints “A” for 65
format	throws an exception

Not allowing ‘c’ to print integer generates a usability problem – the users won’t be able to print the return value of invoking `cin.get()` (also `getc` and `fgetc`) as characters. It is hostile to C++ learners if a cast is required to use `stdio` or `iostreams` with `std::format` for such a trivial task, while “{:c}” can be a way for them to express “show me a character here.”

We propose to let integer presentation types support a new flag, ‘c’, which prints the argument `x` as-if `static_cast<charT>(x)`, where `charT` is the character type of the format string defined in P0645. If the argument is not in the range representable by `charT`, `format_error` is thrown.

Before:

```
int c = 'A';
std::string s = std::format("{:c}", c); // throws format_error
```

After:

```
int c = 'A';
std::string s = std::format("{:c}", c); // s == "A"
```

Problem 3: “-000nan” is not a floating point value

What `printf("%07", -nan(""))` prints is underspecified until C99^[2:1] and POSIX 2008^[3], where the effect of ‘0’ flag is described as:

For **d**, **i**, **o**, **u**, **x**, **X**, **a**, **A**, **e**, **E**, **f**, **F**, **g**, and **G** conversions, leading zeros (following any indication of sign or base) are used to pad to the field width rather than performing space padding, except when converting an infinity or NaN. [...]

The last clause did not present in C89, C90, and POSIX 2003. The output “-000nan” cannot be correctly parsed by `iostreams` and `strtod`. As of 2016, FreeBSD `libc`, `glibc`, and Microsoft `UCRT` have all avoided it.

However, `iostreams` mandates this pathological output with the `internal iomanip`. This output also presents in Python and `fmt` where the `= alignment` type is functionally equivalent to `internal`. Even worse, the dedicated ‘0’ *std-format-spec* is specified as “[...] equivalent to a fill character of ‘0’ with an *alignment* type of ‘=’”. So the output of ‘0’ flag in Python and `fmt` is incompatible with `printf` ‘0’ flag.

The observations are:

1. The `internal iomanip` only affects numeric output and does it poorly;
2. The ‘=’ *alignment* type inherited all issues from `internal` and is verbose to write, hard to interpret, compared to ‘0’.

Therefore, we propose to remove the ‘=’ *alignment* type and respecify ‘0’ to match C99 `printf`’s output. Note that Rust `std::fmt`, a newer Python-like formatting facility, also removed the ‘=’ align spec.^[4]

Before:

```
double nan = std::numeric_limits<double>::quiet_NaN();
std::string s1 = std::format("{:0=6}", nan); // s1 == "000nan"
std::string s2 = std::format("{:06}", nan);  // s2 == "000nan"
```

After:

```
double nan = std::numeric_limits<double>::quiet_NaN();
std::string s1 = std::format("{:0=6}", nan); // throws format_error
std::string s2 = std::format("{:06}", nan);  // s2 == "    nan"
```

Problem 4: bool needs a type format specifier

variant	behavior
printf	does not print <code>bool</code> as “true” or “false”
iostreams	via <code>boolalpha</code> <code>io manip</code>
Python	no type format specifier for <code>bool</code> but empty format specification invokes <code>str()</code> ^[5] which returns “True” or “False”
format	no type format specifier for <code>bool</code> but empty format specification gives “true” or “false”

So `std::format` can only print `bool` *without* a type format specifier, distinguishing it from all other fundamental types and string-like types. We consider ‘s’ flag to be a “Do What I Mean” (DWIM) improvement to this caveat. Note that the `fmt` library supports printing `bool` via `%s` in `printf`-compatible syntax^[6], but did not propose the syntax for standardization.

Before:

```
std::string s = std::format("{:s}", true); // throws format_error
```

After:

```
std::string s = std::format("{:s}", true); // s == "true"
```

Problem 5: double does not roundtrip float

variant	roundtrip double in shortest decimal representation	float behavior
printf	No	<code>float</code> is promoted to <code>double</code>
iostreams	No	<code>float</code> is converted to <code>double</code>
Python	Yes	does not support <code>float32</code>
format	Yes	<code>float</code> is converted to <code>double</code>

Python prints shortest round-trip representations for floating point values by default; so does `std::format` – but not for `float`. Single-precision floating point values roundtrip in their realm and are already supported by `std::to_chars`. We should print a `float` as `float` rather than a long string used for disambiguating the value in `double`'s realm.

Before:

```
std::string s = std::format("{", 3.31f); // s == "3.309999942779541"
```

After:

```
std::string s = std::format("{", 3.31f); // s == "3.31"
```

Wording

The wording is relative to P0645R9.

Modify 19.?.2 [format.string] as follows:

```
format-spec      ::= std-format-spec | custom-format-spec
std-format-spec ::= [[fill] align] [sign] ['#'] ['0'] [width] ['.' precision] [typ
fill            ::= <a character other than '{' or '}'>
align           ::= '<' | '>' | '=' | '^'
sign            ::= '+' | '-' | ' '
width           ::= nonzero-digit [integer] | '{' arg-id '}'
precision       ::= integer | '{' arg-id '}'
type            ::= 'a' | 'A' | 'b' | 'B' | 'c' | 'd' | 'e' | 'E' | 'f' | 'F' |
                  'g' | 'G' | 'n' | 'o' | 'p' | 's' | 'x' | 'X'
```

[...]

Let `charT` be `decltype(fmt)::value_type`.

The meaning of the various alignment options is as follows:

Option	Meaning
'<'	Forces the field to be left-aligned within the available space. This is the default for non-arithmetic types, <code>charT</code> , and <code>bool</code> , unless an integer presentation type is specified.
'>'	Forces the field to be right-aligned within the available space. This is the default for arithmetic types other than <code>charT</code> and <code>bool</code> or when an integer presentation type is specified.
'='	Forces the padding to be placed after the sign or prefix (if any) but before the digits. This is used for printing fields in the form <code>+000000120</code>. This alignment option is only valid for arithmetic types other than <code>charT</code> and <code>bool</code> or when an integer presentation type is specified.
'^'	Forces the field to be centered within the available space by inserting $N / 2$ and $N - N / 2$ fill characters before and after the value respectively, where N is the total number of fill characters to insert.

[Example:

```
char c = 120;
string s0 = format("{:6}", 42);           // s0 == "    42"
string s1 = format("{:6}", 'x');           // s1 == "x     "
string s2 = format("{:<6}", 'x');          // s2 == "x*****"
string s3 = format("{:>6}", 'x');          // s3 == "*****x"
string s4 = format("{:^6}", 'x');          // s4 == "***x***"
string s5 = format("[:=6]", 'x');          // Error: '=' with charT and no integer prese
string s65 = format("{:6d}", c);           // s65 == "   120"
string s7 = format("[:+=6d]", c);          // s7 == "+00120"
string s8 = format("{:0=#6x}", 0xa);       // s8 == "0x000a"
string s96 = format("{:6}", true);         // s96 == "true  "
```

—end example]

The '#' option causes the alternate form to be used for the conversion. This option is only valid for arithmetic types other than `charT` and `bool` or when an integer presentation type is specified. For integers, when binary, ~~octal~~, or hexadecimal output is used, this option adds the respective prefix "0b" ("0B"), ~~"0"~~, or "0x" ("0X") to the output value. Whether the prefix is lower-case or upper-case is determined by the case of the type format specifier. The option prefixes the output value with "0" when octal output is used on nonzero integers. For floating-point numbers [...]

width is a decimal integer defining the minimum field width. If not specified, then the field width will be determined by the content.

Preceding the *width* field by a zero ('0') character ~~enables sign-aware zero-padding for arithmetic types. This is equivalent to a fill character of '0' with an alignment type of '='.~~ pads leading zeros (following any indication of sign or base) to the field width, except when applying to an infinity or NaN. This option is only valid for arithmetic types other than `charT` and `bool` or when an integer presentation type is specified. If the '0' and an *align* option both appear, the '0' character is ignored. *[Example:*

```
char c = 120;  
string s1 = format("{:+06d}", c); // s1 == "+00120"  
string s2 = format("{:#06x}", 0xa); // s2 == "0x000a"
```

~~–end example]~~

[...]

The available integer presentation types and their mapping to `to_chars` are:

Option	Meaning
'b'	<code>to_chars(first, last, value, 2)</code> ; using the '#' option with this type adds the prefix "0b" to the output.
'B'	The same as 'b' , except that the '#' option adds the prefix "0B" to the output.
<u>'c'</u>	<u>Copies the character <code>static cast<charT>(value)</code> to the output. Throws <code>format_error</code> if <code>value</code> is not in the range of representable values for <code>charT</code>.</u>
'd'	<code>to_chars(first, last, value)</code> .
[...]	[...]
none	The same as 'd' if the formatting argument type is not <code>charT</code> or <code>bool</code> .

Integer presentation types can also be used with `charT` and `bool` values. In such a case, a value of type `bool` is treated as `static cast<unsigned char>(value)`. ~~Values of type `bool` are formatted using textual representation, either "true" or "false", if the presentation type is not specified.~~ *[Example: ... –end example]*

[Drafting note: A drive-by fix – `to_chars` has no overload for `bool` . –end note]

Infinity and NaN are formatted as "inf" and "nan" respectively unless specified otherwise.

The available `bool` presentation types are:

Type	Meaning
<u>'s'</u>	<u>Copies textual representation, either "true" or "false", to the output.</u>
<u>none</u>	<u>The same as 's'.</u>

The available pointer presentation types and their mapping to `to_chars` are:

[...]

Modify 19.7.4.1 [format.arg] as follows:

```
namespace std {
    template<class Context>
    class basic_format_arg {
    public:
        class handle;

        using char_type = typename Context::char_type;

        variant<monostate, bool, char_type,
                int, unsigned int, long long int, unsigned long long int,
                float, double, long double,
                const char_type*, basic_string_view<char_type>,
                const void*, handle> value;

        basic_format_arg() noexcept;
    };
};
```

[...]

```
explicit basic_format_arg(float n) noexcept;

Effects: Initializes value with static_cast<double>(n).
```

```
explicit basic_format_arg(double n) noexcept;
explicit basic_format_arg(long double n) noexcept;

Effects: Initializes value with n.
```


References

1. Text Formatting. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0645r9.html>
(<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0645r9.html>) ↩
2. ISO/IEC 9899:TC3 Committee Draft. <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1256.pdf> (<http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1256.pdf>) ↩ ↩
3. `dprintf`, `fprintf`, `printf`, `snprintf`, `sprintf` - print formatted output, The Open Group Base Specifications Issue 7, 2018 edition.
<http://pubs.opengroup.org/onlinepubs/9699919799/functions/fprintf.html>
(<http://pubs.opengroup.org/onlinepubs/9699919799/functions/fprintf.html>) ↩
4. Syntax, Module `std::fmt`. <https://doc.rust-lang.org/std/fmt/#syntax> (<https://doc.rust-lang.org/std/fmt/#syntax>) ↩
5. Printing boolean values `True/False` with the `format()` method in Python.
<https://stackoverflow.com/questions/23655005/printing-boolean-values-true-false-with-the-format-method-in-python/23666923> (<https://stackoverflow.com/questions/23655005/printing-boolean-values-true-false-with-the-format-method-in-python/23666923>) ↩
6. Formatting `bool` with 's' type specifier should give textual output.
<https://github.com/fmtlib/fmt/issues/224> (<https://github.com/fmtlib/fmt/issues/224>) ↩